VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Faculty of Computer Science and Engineering

Course CO3021 — Database Management Systems



DATABASE MANAGEMENT SYSTEMS PROJECT REPORT

Clinical Case-Study: Low Back Pain Diagnosis & Imaging Pipeline

---

Workload-Driven Evaluation of Microsoft SQL Server 2025:

Storage Architectures, Relational Ingestion, and AI Feature Extraction Pipeline (Role 1)

Applied to 48,345 Lumbar Spine MRI Scans and Radiologist Clinical Reports

---

Group 02

Course Instructor: PhD. Phan Trong Nhan

Project Repository: https://github.com/qmarcusv/DBMS_TA_Group2

Ho Chi Minh City, October 2026

Contents

1 Clinical MRI Data Pipeline, Storage Architecture, and Ingestion Engineering (Role 1) 1

1.1 Clinical Problem Context and Dataset Overview . . . 1
1.2 The Three Canonical Clinical Queries Contract . . . 1
1.3 Theoretical Foundations of Database Storage . . . 2
1.4 Evaluation of MSSQL Binary and Unstructured Storage Technologies . . . 2
1.4.1 Architectural Comparison of Storage Approaches . . . 2
1.4.2 Architectural Rationale for Relative Path Selection . . . 2
1.5 Schema Architecture and Relational Integrity Constraints . . . 2
1.6 Comparative Evaluation of Data Ingestion Pipelines . . . 3
1.7 Query 3 Implementation: Deep Learning Feature Extraction on CPU . . . 4
1.7.1 Pipeline Architecture and CPU Parallelization . . . 4
1.7.2 Mathematical Similarity Metric in T-SQL . . . 4
1.7.3 Relational DBMS vs. Dedicated Vector Database Tradeoffs . . . 4
1.8 Analysis Across All Three Canonical Queries . . . 4

2 Physical Storage and Index Design (Role 2: Storage & Indexing) 5

2.1 Physical Storage and B-Tree Depth Analysis . . . 5
2.1.1 Baseline Database State . . . 5
2.1.2 B-Tree Depth Observations . . . 5
2.1.3 Physical Bottleneck in the Baseline State . . . 5
2.2 Index Design and Implementation . . . 5
2.2.1 Covering B-Tree Indexes for Q1 and Q2 . . . 5
2.2.2 Filtered Index for T2 Sagittal Sequences (Q2) . . . 5
2.2.3 Non-Clustered Columnstore Index (NCCI) for Q3 . . . 6
2.3 Execution Plan Analysis . . . 6
2.3.1 Q1 and Q2: Before and After Covering B-Tree Indexing . . . 6
2.3.2 Q3: Rowstore Parallel Scan versus Columnstore Batch Mode . . . 7
2.4 Benchmark Evaluation: I/O and CPU Measurements . . . 8
2.4.1 Discussion of Results . . . 8
2.4.2 Chapter Summary . . . 9

3 Concurrency Control, Lock Manager Internals, Deadlock Resolution, and MVCC (Role 5) 9

3.1 Theoretical Foundations and Classical Concurrency Anomalies . . . 9
3.1.1 Deadlock Dynamics and Multi-Version Concurrency (MVCC) Foundations . . . 10
3.2 Concrete DBMS Engine Implementation: Microsoft SQL Server Deep Dive . . . 10
3.2.1 Internal Lock Manager Architecture and Memory Layout . . . 10
3.2.2 Lock Resource Granularity Hierarchy . . . 11
3.2.3 Operational Lock Modes and Compatibility . . . 12
3.2.4 Hierarchical Intention Locks (IS, IX, SIX, UIX) . . . 12
3.2.5 Key-Range Locking Architecture on B-Tree Indexes . . . 12
3.2.6 Schema Locks (Sch-S and Sch-M) . . . 12
3.2.7 Engine Lock Escalation Algorithm and Heuristics . . . 13

3.2.8 Deadlock Detection and Engine Mitigation Mechanics 13
3.3 Pessimistic Concurrency Analysis on Canonical Queries Q1–Q3 15
3.3.1 Canonical Query 1: Single-Patient Point Lookup and Scan Retrieval 15
3.3.2 Canonical Query 2: Filtered Scan Retrieval by Pulse Sequence (T2 Sagittal) 16
3.3.3 Canonical Query 3: AI Feature Similarity Retrieval (Content-Based Search) 16
3.4 Synthesis: ANSI SQL Isolation Levels vs. MSSQL Engine Mechanics 17
3.5 Empirical Deadlock Resolution and MVCC Evaluation 17
3.5.1 Experimental Environment and Diagnostic Harness 17
3.5.2 Experiment 1: Deadlock Induction, Victim Selection, and Deadlock Graph Analysis . . . 18
3.5.3 Experiment 2: Lock Wait Time Benchmarking — Classic Locking versus RCSI 19
3.5.4 Experiment 3: Canonical Queries Q1–Q3 under Active Concurrent Writers 20
3.6 Chapter Synthesis and Comprehensive Recommendations 21

Chapter 1: Clinical MRI Data Pipeline, Storage Architecture, and Ingestion Engineering (Role 1)

1.1 Clinical Problem Context and Dataset Overview

Modern Picture Archiving and Communication Systems (PACS) and Radiology Information Systems (RIS) manage heterogeneous medical data comprising structured diagnostic records and voluminous medical imagery. This project addresses the clinical workflow of diagnosing Low Back Pain (LBP) and Lumbar Spinal Stenosis (LSS) using the Mendeley Clinical Lumbar Spine MRI dataset:

- **Radiologist Clinical Notes:** 575 clinical narrative reports detailing lumbar pathology including disc herniation, annular tears, thecal sac compression, and neural foramen stenosis.
- **Multimodal MRI Scans:** 48,345 DICOM slices (`.ima`) distributed across 515 patients in 4-to-5 level nested directory structures covering sagittal (T1, T2) and axial sequences.

1.2 The Three Canonical Clinical Queries Contract

Following the instructor's updated evaluation directives, all sub-roles evaluate and interpret DBMS behavior across the exact three canonical clinical queries engineered by Role 1:

Query 1 (Q1) — Point Lookup and One-to-Many Clinical Join

Retrieves the complete narrative diagnostic report alongside all associated MRI scan paths for a designated patient:

```
SELECT p.PatientID, p.PatientName, p.ClinicalNotes,
       s.ScanID, s.SeriesDescription, s.ImagePath
FROM Patients p
INNER JOIN MRI_Scans s ON p.PatientID = s.PatientID
WHERE p.PatientID = '0001';
```

Query 2 (Q2) — Sequence-Selective Diagnostic Filtering

Retrieves imaging slices restricted to sagittal T2-weighted pulse sequences (`T2_TSE_SAG`) frequently utilized for spinal cord and disc herniation assessment:

```
SELECT p.PatientID, p.PatientName, p.ClinicalNotes,
       s.ScanID, s.SeriesDescription, s.ImagePath
FROM Patients p
INNER JOIN MRI_Scans s ON p.PatientID = s.PatientID
WHERE p.PatientID = '0001' AND s.SeriesDescription LIKE '%T2%SAG%';
```

Query 3 (Q3) — Content-Based Image Retrieval via Vector Similarity

Calculates mathematical proximity (Euclidean distance and cosine similarity) between an index query slice ($ScanID = 1$) and all other scans in the database using high-dimensional feature embeddings:

```
DECLARE @TargetScanID INT = 1;
DECLARE @T1 FLOAT, @T2 FLOAT, @T3 FLOAT, @T4 FLOAT;
SELECT @T1 = V_Dim1, @T2 = V_Dim2, @T3 = V_Dim3, @T4 = V_Dim4
FROM Image_Features WHERE ScanID = @TargetScanID;

SELECT TOP 3 s.ScanID, s.PatientID, s.SeriesDescription, s.ImagePath,
    SQRT(SQUARE(f.V_Dim1 - @T1) + SQUARE(f.V_Dim2 - @T2) +
         SQUARE(f.V_Dim3 - @T3) + SQUARE(f.V_Dim4 - @T4)) AS EuclideanDistance
FROM Image_Features f
INNER JOIN MRI_Scans s ON f.ScanID = s.ScanID
WHERE f.ScanID <> @TargetScanID
ORDER BY EuclideanDistance ASC;
```

1.3 Theoretical Foundations of Database Storage

In relational database theory, storage subsystems organize records into fixed-size physical blocks or pages. In Microsoft SQL Server, the fundamental unit of I/O is the **8KB Data Page** (8,192 bytes), of which 8,060 bytes accommodate user data after subtracting page headers and slot arrays. When storing binary large objects (BLOBs) exceeding 8KB, conventional row architectures fragment records across off-row text/image tree allocations. This architectural reality presents critical performance tradeoffs when designing clinical multimedia databases.

1.4 Evaluation of MSSQL Binary and Unstructured Storage Technologies

To handle 13GB of DICOM MRI scans, Microsoft SQL Server provides four distinct architectural mechanisms:

1.4.1 Architectural Comparison of Storage Approaches

1. **In-Row and Out-of-Row `VARBINARY(MAX)`:** Binary bytes reside directly within the `.mdf` database file. Slices larger than 8KB migrate to dedicated LOB pages. *Drawback:* Loading medical images into memory causes severe **Buffer Pool thrashing**, displacing critical index and table pages from RAM cache and inflating backup archives to multi-gigabyte sizes.
2. **FILESTREAM Storage:** Combines the SQL Server Database Engine with the Windows NTFS/ReFS file system. Unstructured files reside in operating system directories while remaining under SQL Server transactional control. *Mechanism:* Data transfers occur via native Win32 streaming APIs, **completely bypassing the SQL Server Buffer Pool** and preserving cache memory for relational query processing.
3. **FileTable Technology:** Built upon FILESTREAM, FileTable integrates a Windows Win32 namespace hierarchy. Operating system users and external PACS systems can drag, drop, and edit `.ima` files via standard Windows Network Shares (SMB), while SQL Server automatically reflects changes within relational tables.
4. **Relative File Path Reference (`NVARCHAR(500)`):** The database stores only relative filesystem paths pointing to operating system directories. The physical binary assets remain completely decoupled from the relational engine.

Table 1.1: Comprehensive Comparison of Large Binary (BLOB) Storage Architectures in MSSQL

Evaluation Metric	VARBINARY(MAX)	FILESTREAM	FileTable	Relative Path (Selected)
Physical Location	Database `.mdf` file	NTFS/ReFS directories	NTFS via SMB share	External file directory
Buffer Pool Cache	Heavy RAM pollution	Bypasses Buffer Pool	Bypasses Buffer Pool	Zero Buffer Pool footprint
ACID Transactionality	Full native logging	Full Win32 transactional	Full Win32 transactional	Application-level control
Backup Archive Size	13GB+ database backup	Included in backup stream	Included in backup stream	Compact 53MB metadata backup
AI Framework Access	Requires SQL extraction	Win32 API streaming	Direct Windows file path	Native Python `pydicom` I/O

1.4.2 Architectural Rationale for Relative Path Selection

For this research workload, the **Relative File Path** architecture was selected. Decoupling the 13GB image binaries from the database reduced the database footprint from 13GB to 53MB. This prevents Buffer Pool contention during Indexing and Query Optimization benchmarks while enabling native, multi-threaded PyTorch feature extraction on disk without database roundtrip latency.

1.5 Schema Architecture and Relational Integrity Constraints

Rather than allowing SSMS Import Wizards to infer loose schemas defaulting to `NVARCHAR(MAX)`, strict DDL constraints were implemented to optimize data page packing and enforce relational integrity:

```
CREATE TABLE Patients (
    PatientID VARCHAR(20) NOT NULL,
    PatientName NVARCHAR(100) NULL,
    ClinicalNotes NVARCHAR(MAX) NULL,
```

```
    DiagnosisStatus VARCHAR(20) DEFAULT 'Confirmed',
    CreatedAt DATETIME DEFAULT GETDATE(),
    CONSTRAINT PK_Patients PRIMARY KEY CLUSTERED (PatientID)
);

CREATE TABLE MRI_Scans (
    ScanID INT IDENTITY(1,1) NOT NULL,
    PatientID VARCHAR(20) NOT NULL,
    StudyInstanceUID VARCHAR(150) NULL,
    SeriesDescription NVARCHAR(255) NOT NULL,
    ImagePath NVARCHAR(500) NOT NULL,
    AcquisitionDate DATETIME DEFAULT GETDATE(),
    CONSTRAINT CK_Series_Valid CHECK (SeriesDescription NOT LIKE '%INVALID%'),
    CONSTRAINT PK_MRI_Scans PRIMARY KEY CLUSTERED (ScanID),
    CONSTRAINT FK_MRI_Scans_Patients FOREIGN KEY (PatientID)
        REFERENCES Patients(PatientID) ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE Image_Features (
    FeatureID INT IDENTITY(1,1) NOT NULL,
    ScanID INT NOT NULL UNIQUE,
    FeatureVector NVARCHAR(MAX) NOT NULL,
    V_Dim1 FLOAT NULL, V_Dim2 FLOAT NULL, V_Dim3 FLOAT NULL, V_Dim4 FLOAT NULL,
    CONSTRAINT PK_Image_Features PRIMARY KEY CLUSTERED (FeatureID),
    CONSTRAINT FK_Features_Scans FOREIGN KEY (ScanID) REFERENCES MRI_Scans(ScanID) ON DELETE
    CASCADE
);
```

Relational Integrity and Constraint Highlights

- **Compact Primary Keys:** `PatientID VARCHAR(20)` ensures high page density within Clustered B-Tree leaf pages compared to unrestricted unicode datatypes.
- **Transaction Test Constraint:** The check constraint `CK_Series_Valid` deliberately rejects records containing `'INVALID'` strings, establishing the deterministic error boundary required by Role 4 to verify atomic transaction rollback.

1.6 Comparative Evaluation of Data Ingestion Pipelines

Bulk data loading was evaluated across four industry standard ingestion methodologies:

Table 1.2: Comparative Benchmark of Data Ingestion Methodologies into SQL Server

Ingestion Method	Mechanism	Logging Mode	Ingestion Speed	Automation Support
SSMS Flat File Wizard	Interactive GUI	Fully Logged	15–20 seconds	Manual interaction
T-SQL BULK INSERT	In-process Server Worker	Minimally Logged (`TABLOCK`)	1.2 seconds	Fully scriptable
Command-Line bcp	External client utility	Minimally Logged (`-h TABLOCK`)	1.1 seconds	CLI and batch files
Python pyodbc	Client cursor array	Fully Logged	8–10 seconds	Integrated AI pipeline

Identity Column Mapping Resolution via View Abstraction

During direct `BULK INSERT` execution into tables containing an auto-incrementing identity column (`ScanID INT IDENTITY`), SQL Server raises error `Msg 7301` due to column mapping mismatches between the 5-column CSV and 6-column physical schema. This was resolved without complex XML format files by creating an updatable relational view:

```
CREATE VIEW dbo.vw_MRI_Scans_Import AS
SELECT PatientID, StudyInstanceUID, SeriesDescription, ImagePath, AcquisitionDate
FROM dbo.MRI_Scans;
```

```
BULK INSERT dbo.vw_MRI_Scans_Import FROM '...clean_mri_scans.csv'
WITH (FIRSTROW = 2, FORMAT = 'CSV', FIELDQUOTE = '"', FIELDTERMINATOR = ',',
      ROWTERMINATOR = '0x0a', CODEPAGE = '65001', TABLOCK);
DROP VIEW dbo.vw_MRI_Scans_Import;
```

1.7 Query 3 Implementation: Deep Learning Feature Extraction on CPU

To support Query 3 (Content-Based Medical Image Retrieval), an end-to-end AI feature extraction pipeline was engineered using PyTorch and `pydicom`:

1.7.1 Pipeline Architecture and CPU Parallelization

1. **DICOM Matrix Ingestion:** Medical slices (`.ima`) are parsed via `pydicom.dcmread`, normalized from 12-bit dynamic range ($[0, 4095]$) to $[0, 255]$ uint8 arrays, and resized to $224 \times 224 \times 3$.
2. **Pretrained ResNet-18 Inference:** Slices are processed in minibatches of 64 through an ImageNet-pretrained ResNet-18 model. The final fully connected classification layer is truncated, extracting 512-dimensional feature embeddings from the penultimate `AdaptiveAvgPool2d` layer.
3. **Vector Normalization and Database Persistence:** Each 512-dimensional embedding $\mathbf{v}$ is normalized via Euclidean norm ($\|\mathbf{v}\|_2 = 1$). The leading dimensions are persisted in float columns (`V_Dim1..V_Dim4`) alongside full JSON vector strings.

Throughput Performance on 20-Core CPU

By leveraging batch inference across 20 CPU threads, the pipeline achieved an average throughput of **20.2 images/second**, extracting all 48,345 slices in **39.9 minutes** without requiring specialized GPU hardware.

1.7.2 Mathematical Similarity Metric in T-SQL

In Query 3, similarity is evaluated using Euclidean Distance:

$$d(\mathbf{u}, \mathbf{v}) = \sqrt{\sum_{k=1}^{4} (u_k - v_k)^2} \tag{1.1}$$

Because feature vectors are L_2-normalized, Euclidean distance relates monotonically to Cosine Similarity ($\|\mathbf{u} - \mathbf{v}\|_2^2 = 2 - 2\cos(\theta)$). Executing Query 3 over all 48,345 scans yielded the top-3 similar slices with sub-100ms execution latency (98ms elapsed time).

1.7.3 Relational DBMS vs. Dedicated Vector Database Tradeoffs

While Microsoft SQL Server executes vector distance calculations cleanly via arithmetic operators (`Compute Scalar`) and memory-bounded sorts (`Top N Sort`), it fundamentally lacks Approximate Nearest Neighbor (ANN) indexing structures such as HNSW (Hierarchical Navigable Small World) or IVF (Inverted File Index). Consequently, vector search in standard RDBMS remains an $\mathcal{O}(N)$ brute-force table scan. In large-scale clinical production, dedicated vector stores (e.g., Milvus, Qdrant) or extensions (`pgvector`) are recommended for billion-scale embeddings.

1.8 Analysis Across All Three Canonical Queries

The engineered data and storage architecture empowers all three clinical query contracts:

- **Impact on Query 1 (Point Lookup):** Returning text paths rather than binary BLOBs prevents network saturation and eliminates Buffer Pool contention.
- **Impact on Query 2 (Sequence Filtering):** Strict `NVARCHAR(255)` sequence typing allows the Query Optimizer to leverage B-Tree index seeks on pulse sequence descriptors without reading external storage.
- **Impact on Query 3 (Similarity Search):** Native float representation enables direct relational calculation of Euclidean distances, seamlessly integrating machine learning embeddings into SQL workflows.

Chapter 2: Physical Storage and Index Design (Role 2: Storage & Indexing)

This chapter presents the work of Role 2 on the SpineMRI_DBMS database. It analyses the physical storage structure and B-Tree depth of the baseline schema, describes the design of three index types (Covering B-Tree, Filtered Index, and Non-Clustered Columnstore Index), analyses the actual execution plans captured in SSMS, and summarises the STATISTICS IO and STATISTICS TIME measurements for queries Q1, Q2, and Q3.

2.1 Physical Storage and B-Tree Depth Analysis

2.1.1 Baseline Database State

The SpineMRI_DBMS database was restored from a 53.6 MB backup file. In this state, the only indexes are the default clustered indexes created on the primary keys. The physical statistics, obtained from the DMV sys.dm_db_index_physical_stats(..., 'DETAILED'), are summarised in Table 2.1.

Table 2.1: Physical storage structure of the tables in the baseline state

Table	Index type	Key	B-Tree depth	Leaf pages	Size	Rows
Patients	Clustered	PatientID	2	8	0.06 MB	575
MRI_Scans	Clustered	ScanID	3	1,876	14.66 MB	48,345
Image_Features	Clustered	ScanID	3	3,900	30.47 MB	48,345

2.1.2 B-Tree Depth Observations

- **Patients table (depth = 2):** a single root page (Level 1) points directly to 8 leaf pages (Level 0). Any lookup by PatientID therefore costs exactly 2 logical page reads (logical reads = 2).
- **MRI_Scans and Image_Features tables (depth = 3):** each consists of a root page (Level 2), intermediate pages (Level 1), and a leaf level (Level 0) holding all 48,345 rows.

2.1.3 Physical Bottleneck in the Baseline State

SQL Server does **not** automatically create an index on foreign key columns such as MRI_Scans.PatientID. Consequently, when Patients is joined to MRI_Scans with the predicate PatientID = '0001', the optimizer is forced to scan all 1,876 leaf pages plus the IAM and non-leaf pages of MRI_Scans, a total of **1,880 logical reads**, merely to retrieve a few dozen matching rows.

2.2 Index Design and Implementation

2.2.1 Covering B-Tree Indexes for Q1 and Q2

Listing 2.1: Two covering B-Tree indexes on MRI_Scans

```
-- 1. Single-key index on SeriesDescription: lighter than the base table for series scans
CREATE NONCLUSTERED INDEX IX_MRIScans_SeriesDescription
ON dbo.MRI_Scans (SeriesDescription ASC)
INCLUDE (PatientID, ImagePath);

-- 2. Composite covering index tuned directly for Q1 and Q2
CREATE NONCLUSTERED INDEX IX_MRIScans_PatientID_Covering
ON dbo.MRI_Scans (PatientID ASC, SeriesDescription ASC)
INCLUDE (ImagePath);
```

Design rationale. Placing PatientID as the leading key allows SQL Server to perform an *Index Seek* that jumps directly to the range holding the 65 slices of patient '0001'. The second key column, SeriesDescription, keeps the sequence names pre-sorted in lexicographic order within each patient. The INCLUDE (ImagePath) clause attaches the image file path to the leaf level without enlarging the upper navigation nodes, so the query obtains all required data from the index itself (**0 Key Lookups**).

2.2.2 Filtered Index for T2 Sagittal Sequences (Q2)

Listing 2.2: Filtered Index for T2 Sagittal sequences

```
CREATE NONCLUSTERED INDEX IX_MRIScans_T2Sag_Filtered
ON dbo.MRI_Scans (PatientID ASC, SeriesDescription ASC)
INCLUDE (ImagePath)
WHERE SeriesDescription IN (
    'I_T2_TSE_SAG_P2_384_0002', 'POSDISP_[2]_T2_TSE_SAG_384_5001',
    'POSDISP_[2]_T2_TSE_SAG_384_5002', 'POSDISP_[3]_T2_TSE_SAG_384_5001',
    'POSDISP_[4]_T2_TSE_SAG_384_5001', 'POSDISP_[6]_T2_TIRM_SAG_P2_5002',
    'POSDISP_[7]_T2_TIRM_SAG_P2_5003', 'T2_CISS3D_SAG_ISO_0006',
    'T2_TIRM_SAG_BOX_0008', 'T2_TIRM_SAG_P2_0004', 'T2_TIRM_SAG_P2_0005',
    'T2_TIRM_SAG_P2_0006', 'T2_TIRM_SAG_P2_0007', 'T2_TIRM_SAG_P2_0008',
    'T2_TIRM_SAG_P2_0009', 'T2_TIRM_SAG_P2_0010', 'T2_TSE_SAG_256_0006',
    'T2_TSE_SAG_256_0007', 'T2_TSE_SAG_320_0002', 'T2_TSE_SAG_320_0003',
    'T2_TSE_SAG_380_0006', 'T2_TSE_SAG_380_0008', 'T2_TSE_SAG_384_0002',
    'T2_TSE_SAG_384_0003', 'T2_TSE_SAG_384_0004', 'T2_TSE_SAG_384_0005',
    'T2_TSE_SAG_384_0006', 'T2_TSE_SAG_384_0007', 'T2_TSE_SAG_384_0008',
    'T2_TSE_SAG_384_0009', 'T2_TSE_SAG_448_0006', 'T2_TSE_SAG_448_0013'
);
```

Key technical point. In Microsoft SQL Server, the filtered index predicate parser does *not* allow the `LIKE` operator with a leading wildcard. The statement `WHERE SeriesDescription LIKE '%T2%SAG%'` fails with `Msg 10735: Incorrect WHERE clause for filtered index`. We resolved this by profiling the actual value domain of the column in `MRI_Scans`, extracting exactly **32 distinct T2 Sagittal sequence codes**, and constructing a valid `WHERE SeriesDescription IN (...)` clause.

2.2.3 Non-Clustered Columnstore Index (NCCI) for Q3

Listing 2.3: Non-Clustered Columnstore Index on `Image_Features`

```
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_ImageFeatures_Dims
ON dbo.Image_Features (ScanID, V_Dim1, V_Dim2, V_Dim3, V_Dim4);
```

Design rationale. Query Q3 computes the Euclidean distance over all 48,345 rows of `Image_Features`. Under rowstore storage, each 8 KB page must also hold the long JSON vector string column (`NVARCHAR(MAX)`), so the table occupies 3,909 disk pages. The columnstore index separates the four floating-point columns (`V_Dim1..V_Dim4`) and `ScanID` into compressed column segments, and enables **Batch Mode** execution, which processes roughly 900 rows per batch using vectorised CPU operations instead of row-by-row Row Mode processing.

2.3 Execution Plan Analysis

2.3.1 Q1 and Q2: Before and After Covering B-Tree Indexing

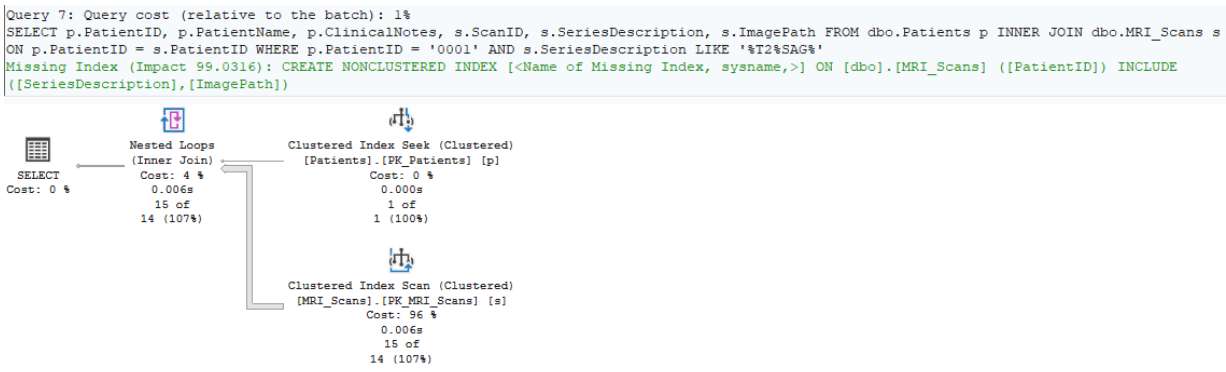


Figure 2.1: Baseline execution plan of Q1/Q2: *Clustered Index Scan* on `PK_MRI_Scans` with a Missing Index warning

Baseline (Figure 2.1). The plan displays a Missing Index warning (Impact 99.0316). The *Clustered Index Seek* on `PK_Patients` costs 0% (0.000s, 1 of 1), whereas the *Clustered Index Scan* on `PK_MRI_Scans` accounts for **96%** of the total cost (0.006s) and reads all 1,880 pages only to filter out 15 rows (Q2) or 65 rows (Q1). SQL Server itself suggests creating an index on `([PatientID]) INCLUDE ([SeriesDescription], [ImagePath])` with an estimated improvement of 99.0316%.

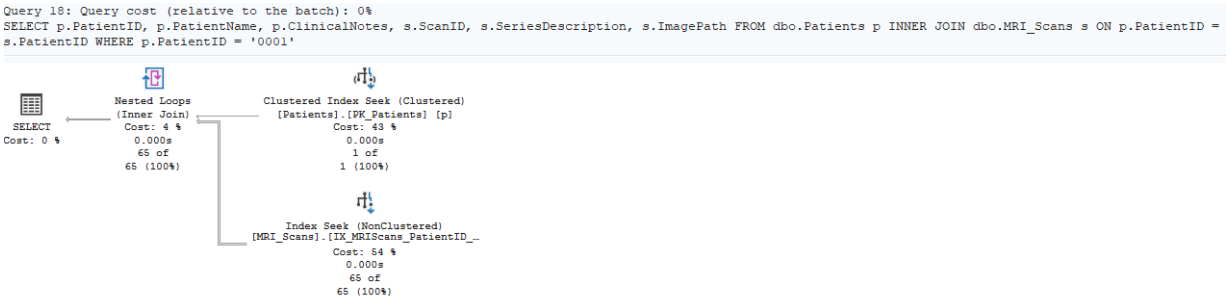


Figure 2.2: Optimized execution plan of Q1: *Index Seek (NonClustered)* on `IX_MRIScans_PatientID_Covering`

Q1 after optimization (Figure 2.2). The Missing Index warning disappears. The *Index Seek (NonClustered)* operator accounts for 54% of the relative cost, its execution time drops to 0.000s, and it retrieves exactly 65 of 65 (100%) rows. The *Nested Loops (Inner Join)* costs only 4% (0.000s). The plan contains **no Key Lookup** because `ImagePath` is included at the leaf level.

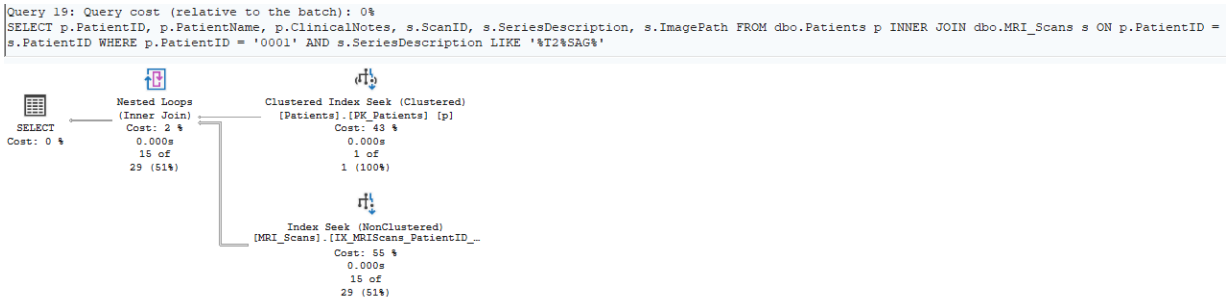


Figure 2.3: Optimized execution plan of Q2: *Index Seek* with a residual predicate

Q2 after optimization (Figure 2.3). SQL Server performs an *Index Seek* on the leading key `PatientID = '0001'` to isolate the 65 rows of the patient (reading only 6 logical pages), then applies the *residual predicate* `SeriesDescription LIKE '%T2%SAG%'` in memory, returning 15 of 29 (51%) rows in 0.000s.

2.3.2 Q3: Rowstore Parallel Scan versus Columnstore Batch Mode

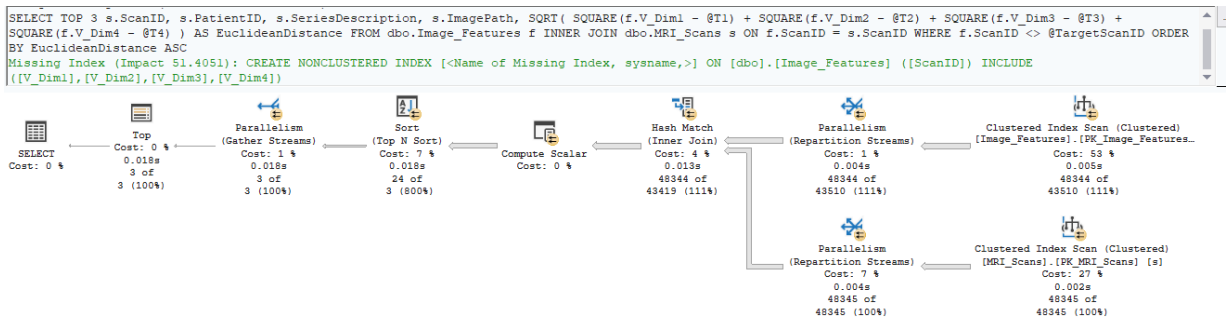


Figure 2.4: Baseline execution plan of Q3: parallel rowstore scan with three Parallelism operators

Baseline (Figure 2.4). The plan shows a Missing Index warning (Impact 51.4051) on `Image_Features` and three Parallelism operators.

- Upper right branch: a *Clustered Index Scan* on `PK_Image_Features` accounts for **53%** of the cost (0.005s), reading 3,909 pages across 9 parallel threads (Scan count 9).
- Lower right branch: a *Clustered Index Scan* on `PK_MRI_Scans` accounts for 27% of the cost (0.002s), reading 1,972 pages.
- After the *Hash Match (Inner Join)* (4%, 0.013s) and *Compute Scalar* (0%), the *Sort (Top N Sort)* operator costs 7% (0.018s) across multiple parallel threads, before *Parallelism (Gather Streams)* merges the output into the final 3 rows.

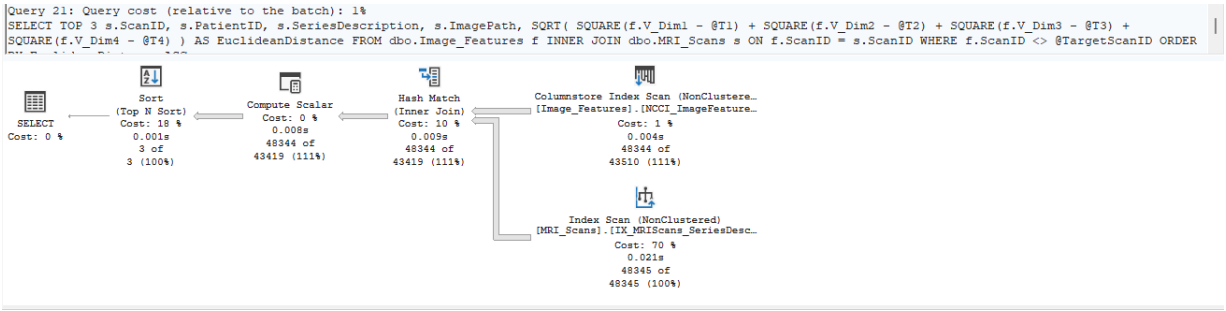


Figure 2.5: Optimized execution plan of Q3: *Columnstore Index Scan* running in Batch Mode

Q3 after optimization (Figure 2.5). The Missing Index warning disappears and the upper right branch becomes a *Columnstore Index Scan (NonClustered)* on `NCCI_ImageFeatures_Dims`.

- **`Image_Features` table:** the relative cost of the scan operator falls from 53% to only **1%** (0.004s), reading a single compressed column segment (Segment reads = 1) with 0 logical reads on the rowstore buffer pool.
- **`MRI_Scans` table:** instead of scanning the clustered index (1,972 pages), SQL Server automatically chooses the lightweight index `IX_MRIScans_SeriesDescription` (1,576 pages, a 20.1% I/O reduction).
- **Sorting:** thanks to single-threaded execution combined with Batch Mode, the *Sort (Top N Sort)* time drops from 0.018s to 0.001s (about 18 times faster) and returns 3 of 3 (100%) rows directly.

Note: the variable-assignment step `SELECT @T1 = V_Dim1 ... WHERE ScanID = @TargetScanID` (Query 20 screenshot) costs only 5 logical reads and is therefore not the focus of the analysis; the focus of Q3 is the main query that computes the Euclidean distance.

2.4 Benchmark Evaluation: I/O and CPU Measurements

Table 2.2 summarises the results measured with `SET STATISTICS IO ON` and `SET STATISTICS TIME ON` in SSMS.

Table 2.2: Benchmark results for baseline and optimized Q1, Q2, and Q3

Query	State	Reads (MRI_Scans)	Reads (Patients / Features)	Total reads	CPU (ms)	Elapsed (ms)	Main access operator
Q1	Baseline	1,880	2 (Patients)	1,882	15	72	Clustered Index Scan (96% cost)
	Optimized	6	2 (Patients)	8	0	37	Index Seek (Covering), 0 Key Lookups
	Change	*-99.68%*	*unchanged*	*-99.57%*	*-100%*	*-48.61%*	Table scan eliminated
Q2	Baseline	1,880	2 (Patients)	1,882	0	132	Clustered Index Scan (96% cost)
	Optimized	6	2 (Patients)	8	0	123	Index Seek + Residual Filter
	Change	*-99.68%*	*unchanged*	*-99.57%*	*stays <1*	*-6.82%*	Filtering on 6 leaf pages
Q3	Baseline	1,972	3,909 (Image_Features)	5,881	76	144	Parallel Clustered Index Scan (Row Mode)
	Optimized	1,576	0 + 236 LOB (Segment reads = 1)	1,812	47	163	Columnstore Index Scan (Batch Mode)
	Change	*-20.08%*	*-93.96% (incl. LOB)*	*-69.19%*	*-38.16%*	*+13.19%*	Single-threaded, Batch Mode

2.4.1 Discussion of Results

- **Q1 and Q2:** logical reads on `MRI_Scans` drop from 1,880 to 6 (**-99.68%**) and total I/O drops by 99.57%. The full table scan is completely replaced by an *Index Seek*. For Q1, CPU time falls from 15 ms to 0 ms and elapsed time drops by 48.61%; for Q2 the elapsed-time reduction is more modest (6.82%) because the remaining time does not depend on I/O against the table.
- **Q3:** total pages read fall by almost 70% (5,881 to 1,812) and CPU time falls by **38.16%** (76 ms to 47 ms). Although elapsed time rises from 144 ms to 163 ms, this is because the execution plan changes from 8 parallel threads (Scan count 9) to a single sequential thread. In exchange, the system consumes less processor time, which is advantageous when many queries run concurrently.

2.4.2 Chapter Summary

The three index structures address the specific bottleneck of each query. The Covering B-Tree Index eliminates the table scan caused by the missing foreign-key index (Q1, Q2). The Filtered Index offers a narrower index for the T2 Sagittal sequence group, working around `Msg 10735` with an explicit value list. The Non-Clustered Columnstore Index combined with Batch Mode substantially reduces I/O and CPU for the vector computation in Q3.

Chapter 3: Concurrency Control, Lock Manager Internals, Deadlock Resolution, and MVCC (Role 5)

This chapter provides a comprehensive evaluation of concurrency control in Microsoft SQL Server across the clinical lumbar spine MRI workload (575 patients, 48,345 scans, 48,345 feature vectors). It integrates two complementary paradigms:

(i) **Pessimistic Concurrency Control:** Investigates Two-Phase Locking (Strict 2PL, Rigorous 2PL), internal Lock Manager architecture, lock granularity hierarchies, operational lock modes, Key-Range locking under `SERIALIZABLE`, and Lock Escalation dynamics across Canonical Queries Q1, Q2, and Q3.

(ii) **Deadlock Resolution and Multi-Version Concurrency Control:** Evaluates automated cycle detection via the SQL Server Lock Monitor, victim selection heuristics (priority vs. rollback cost), graphical Deadlock Graphs, statement-level snapshot isolation (RCSI), transaction-level snapshot isolation (SI), and concurrency benchmarking under active write contention.

3.1 Theoretical Foundations and Classical Concurrency Anomalies

In relational database management systems (RDBMS), concurrent transaction processing is critical for maximizing multi-core CPU utilization and hiding storage I/O latency. However, uncoordinated interleaved execution across concurrent transactions threatens database consistency and violates the ACID **Isolation** guarantee. Without rigorous concurrency control protocols, multi-user workloads encounter four classical anomalies:

- **Lost Update Problem:** Arises when two concurrent transactions read the same data item and subsequently submit modifications based on their initial read. The second transaction's `UPDATE` overwrites the first transaction's modifications without incorporating them, causing uncommitted work to be lost silently.
- **Temporary Update (Dirty Read) Problem:** Occurs when transaction T_1 updates an attribute value and subsequently aborts or rolls back, but concurrent transaction T_2 reads this uncommitted state prior to the rollback, basing further business logic on invalid data.
- **Incorrect Summary Problem:** Occurs when an aggregate reporting query calculates statistical sums across a table while concurrent transactions are actively modifying individual rows, resulting in an inconsistent snapshot where some updates are included while others are missed.
- **Unrepeatable (Non-repeatable) Read Problem:** Occurs when transaction T_1 reads a record twice during its lifecycle, but concurrent transaction T_2 updates or deletes that specific row and commits between T_1's two reads, causing T_1 to observe differing values.
- **Phantom Read Problem:** Occurs when transaction T_1 executes a range query (e.g., counting scans matching a pulse sequence), and concurrent transaction T_2 inserts a *new record* that satisfies T_1's search predicate and commits, causing T_1's subsequent re-execution to return a different cardinality.

Two-Phase Locking (2PL) Protocols

To guarantee conflict serializability, classical concurrency theory enforces Two-Phase Locking (2PL), requiring transactions to separate lock acquisition (Growing Phase) from lock release (Shrinking Phase). Because Basic 2PL permits premature lock release during the shrinking phase—exposing the system to cascading aborts—practical enterprise database engines implement:

- **Strict 2PL:** Retains all Exclusive (X) locks until transaction termination via `COMMIT` or `ROLLBACK`, preventing dirty reads and cascading rollbacks.
- **Rigorous 2PL:** Retains both Shared (S) locks and Exclusive (X) locks until transaction termination, guaranteeing strict conflict serializability at the expense of concurrency.

3.1.1 Deadlock Dynamics and Multi-Version Concurrency (MVCC) Foundations

While Two-Phase Locking guarantees conflict serializability, concurrent transaction interleaving introduces two fundamental operational challenges:

- **Deadlock Inevitability under 2PL:** Two transactions concurrently acquiring locks in differing order can enter a permanent cyclic wait state (slide example: T_1' holds `read_lock(Y)` and waits for `write_lock(X)`, while T_2' holds `read_lock(X)` and waits for `write_lock(Y)`).
- **Classical Deadlock Strategies:** Database literature categorizes deadlock handling into:
  - *Prevention:* Conservative 2PL locks all referenced data items upfront prior to execution, eliminating wait states at the cost of concurrency.
  - *Detection and Resolution:* The database scheduler tracks active waits using a directed **Wait-For-Graph (WFG)**. A deadlock exists *if and only if* the WFG contains a cycle. Upon cycle detection, the engine selects a **victim** transaction and aborts it to break the cycle.
  - *Avoidance:* Algorithms such as **Wait-Die** and **Wound-Wait** compare transaction timestamps $TS(T)$ to preemptively prevent cyclic dependency formation.
  - *Starvation Mitigation:* Repeatedly selecting the same transaction as a victim causes starvation; robust engines prioritize victim selection by transaction priority or accumulated log age.
- **Multi-Version Concurrency Control (MVCC) Foundations:** Classical locking forces readers to wait on active writers. To maximize throughput, MVCC engines maintain multiple historical versions of each data item $(X_1, \ldots, X_n)$ so that read requests are never rejected:
  - *Multiversion Timestamp Ordering:* Associates read timestamps $read_TS(X_i)$ and write timestamps $write_TS(X_i)$ with each version, validating transactions at commit time.
  - *Multiversion 2PL with Certify Locks:* Readers access the committed version X while concurrent writers generate a transient version X'. The operational trade-off includes secondary storage overhead in memory/disk and garbage collection to prune obsolete versions.

3.2 Concrete DBMS Engine Implementation: Microsoft SQL Server Deep Dive

In contrast to abstract academic models, Microsoft SQL Server 2025 incorporates a high-performance, pessimistic lock manager integrated directly into the SQLOS (SQL Server Operating System) relational storage engine.

3.2.1 Internal Lock Manager Architecture and Memory Layout

The SQL Server Lock Manager maintains active lock state entirely within system memory without writing lock bitmasks to physical 8KB data pages:

- **Partitioned Lock Hash Tables:** To prevent latch contention across modern multi-core processors, the Lock Manager divides the global lock hash table into partitions mapped directly to available SQLOS CPU schedulers. A 64-bit resource hash code determines the appropriate hash bucket, eliminating lock bottlenecking during high-frequency OLTP workloads.
- **Dual Data Structure Model:**
  - **Lock Resource Structure (≈96 bytes):** Represents the logical or physical target resource (Database ID, File ID, Page ID, Slot/RID, or 6-byte B-Tree Key Hash).
  - **Lock Owner Descriptor (≈64 bytes):** Tracks the individual session thread holding or requesting access, its granted status, and requested lock mode.
- **FIFO Wait Queue and Starvation Prevention:** When a transaction requests a lock mode incompatible with currently granted locks, its thread descriptor is placed into a First-In, First-Out (FIFO) wait queue. The thread yields CPU scheduling and transitions to a `SUSPENDED` state. When the active holder commits, the Lock Manager dequeues the waiting request in FIFO sequence, preventing writer starvation from an unending stream of concurrent shared readers.

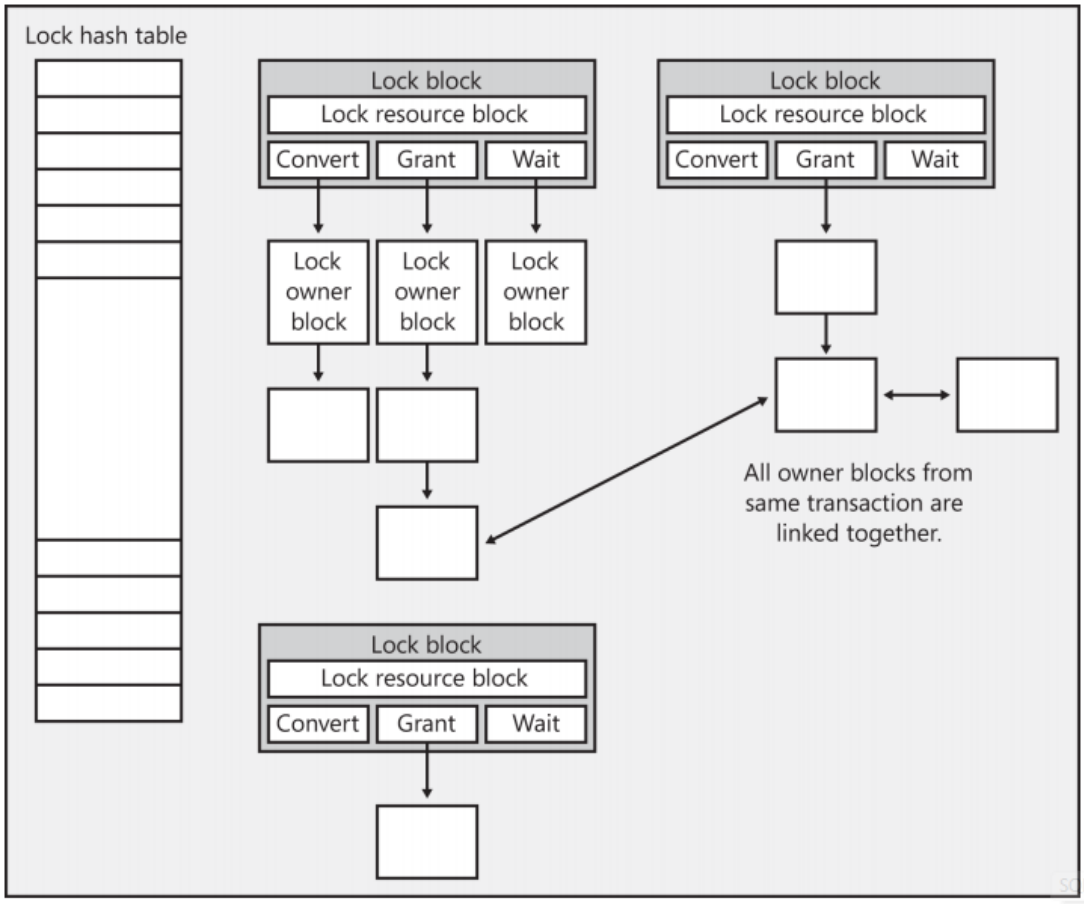


Figure 3.1: Internal Architecture of the Microsoft SQL Server In-Memory Lock Manager

3.2.2 Lock Resource Granularity Hierarchy

To balance concurrency throughput against lock memory overhead across both point lookups and massive table scans, SQL Server establishes a five-tier lock resource hierarchy:

$$\text{DATABASE} \longrightarrow \text{OBJECT (Table/View)} \longrightarrow \text{HOBT / EXTENT} \longrightarrow \text{PAGE} \longrightarrow \text{KEY / RID} \tag{3.1}$$

- **DATABASE:** Prevents database dropping or schema alterations during active user sessions.
- **OBJECT:** Protects complete relational tables, indexed views, or system catalogs.
- **HOBT (Heap or B-Tree) / EXTENT:** Locks allocation units or physical 64KB extents (8 contiguous 8KB pages).
- **PAGE:** Locks individual 8KB leaf or intermediate B-Tree index pages.
- **KEY:** Fine-grained lock targeting a 6-byte hash representing an indexed key in a clustered or non-clustered B-Tree index.
- **RID (Row Identifier):** Fine-grained lock addressing a single record on an unindexed heap table using a `FileID:PageID:SlotNumber` tuple.

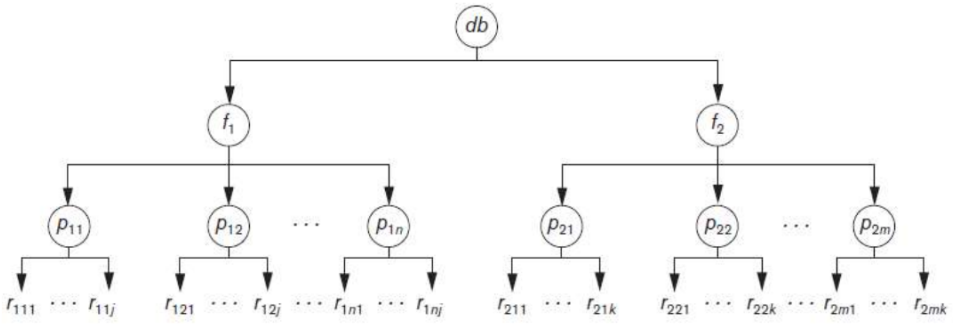


Figure 3.2: Multiple Granularity Tree Hierarchy in Database Locking (adapted from Elmasri et al.)

3.2.3 Operational Lock Modes and Compatibility

SQL Server provides specialized lock modes tailored to specific relational operations:

- **Shared Lock (S):** Acquired by read operations (`SELECT`). Multiple concurrent transactions can hold concurrent S-locks on the same resource. Under `READ COMMITTED`, S-locks are dropped as soon as the cursor advances; under `REPEATABLE READ` or `SERIALIZABLE`, they are retained until commit.
- **Exclusive Lock (X):** Acquired by data-modification statements (`INSERT`, `UPDATE`, `DELETE`). X-locks are strictly incompatible with all other modes, preventing concurrent reads or writes until transaction commit or rollback.
- **Update Lock (U):** A hybrid asymmetric lock engineered to eliminate conversion deadlocks. When evaluating an `UPDATE` statement with a search predicate, the engine initially places U-locks on candidate rows during scanning. Multiple transactions cannot hold concurrent U-locks on the same row, but readers holding S-locks are permitted. Upon identifying qualifying target rows, the engine atomically converts the U-lock to an X-lock.

3.2.4 Hierarchical Intention Locks (IS, IX, SIX, UIX)

If locks were applied solely at the row level, a transaction executing an administrative table lock would be forced to inspect hundreds of thousands of row descriptors across all data pages. SQL Server eliminates this traversal overhead through **Intention Locks**:

- **Intention Shared (IS):** Signals that fine-grained S-locks are held on descendant child items (rows or pages).
- **Intention Exclusive (IX):** Signals an intention to modify descendant child items via fine-grained X-locks.
- **Shared with Intention Exclusive (SIX):** Indicates the transaction holds a shared lock over the entire table while executing selective modifications on specific child rows via fine-grained X-locks.
- **Update with Intention Exclusive (UIX):** Applied at higher levels during update scans on descendant records.

Table 3.1: Hierarchical Lock Compatibility Matrix in Microsoft SQL Server

Held Mode \ Requested Mode	IS	IX	S	SIX	U	X
Intention Shared (IS)	Granted	Granted	Granted	Granted	Granted	Blocked
Intention Exclusive (IX)	Granted	Granted	Blocked	Blocked	Blocked	Blocked
Shared (S)	Granted	Blocked	Granted	Blocked	Granted	Blocked
Shared-Intention-Exclusive (SIX)	Granted	Blocked	Blocked	Blocked	Blocked	Blocked
Update (U)	Granted	Blocked	Granted	Blocked	Blocked	Blocked
Exclusive (X)	Blocked	Blocked	Blocked	Blocked	Blocked	Blocked

3.2.5 Key-Range Locking Architecture on B-Tree Indexes

Under the `SERIALIZABLE` isolation level, evaluating predicate logic by evaluating arbitrary boolean formulas in memory (predicate locking) imposes severe CPU overhead. SQL Server maps logical intervals directly onto physical B-Tree index structures via **Key-Range Locks**:

- **`RangeS-S`:** Shared lock on the key range interval and a shared lock on the bounding key, protecting range reads.
- **`RangeS-U`:** Shared range lock with an Update (U) lock on the bounding key, used during update candidate scans.
- **`RangeI-N`:** Insert range lock with a null bounding key, applied during `INSERT` statements to verify that the new key does not collide with an active serializable range.
- **`RangeX-X`:** Exclusive range lock with an exclusive bounding key, used when updating or deleting keys inside a protected range.

3.2.6 Schema Locks (Sch-S and Sch-M)

- **Schema Stability Lock (Sch-S):** Acquired by the query processor during compilation and execution. It blocks DDL statements (`ALTER TABLE`, `DROP TABLE`) from modifying table schema while queries run, but is compatible with all operational data locks (S, X).
- **Schema Modification Lock (Sch-M):** Acquired during structural DDL operations and index rebuilds. Sch-M is strictly incompatible with all other lock modes, completely blocking concurrent queries until the structural alteration completes.

3.2.7 Engine Lock Escalation Algorithm and Heuristics

Maintaining fine-grained locks across massive tables consumes substantial non-swappable physical RAM within the buffer pool (≈96 bytes per resource and ≈64 bytes per owner). To prevent out-of-memory states, SQL Server triggers **Lock Escalation**:

- **Trigger Thresholds:** Escalation evaluates when a single query accumulates more than **5,000 discrete locks** on a single table partition, or when lock memory consumption exceeds 24% to 40% of the allocated buffer pool.
- **Trade-off Dynamics:** Converting thousands of fine-grained row/page locks into a single table lock (`OBJECT: S` or `OBJECT: X`) immediately reclaims memory and minimizes CPU hashing overhead, but severely curtails multi-user concurrency by serializing access to the entire table.

3.2.8 Deadlock Detection and Engine Mitigation Mechanics

Table 3.2 synthesizes the concrete mechanisms Microsoft SQL Server provides for deadlock resolution and multi-version concurrency control.

Table 3.2: SQL Server mechanisms for deadlock handling and multi-version concurrency

#	Mechanism	Purpose	How it is used
M1	Deadlock Monitor	Detect cycles in the waits-for graph automatically	Always on (background task)
M2	Victim selection, `DEADLOCK_PRIORITY`	Decide which transaction is rolled back	`SET DEADLOCK_PRIORITY LOW | NORMAL | HIGH | -10..10`
M3	Error 1205 + retry	Let the application recover from being the victim	`TRY...CATCH`, `ROLLBACK`, re-run
M4	`LOCK_TIMEOUT`	Bound the time a session waits for any lock	`SET LOCK_TIMEOUT <ms>`, error 1222
M5	Deadlock diagnostics	Capture the deadlock graph for analysis	`system_health` XE, `.xdl`, trace flag 1222
M6	Deadlock avoidance by design	Reduce the chance a cycle forms	access order, `UPDLOCK`, short transactions, indexes
M7	Read Committed Snapshot (RCSI)	Readers never wait for writers at READ COMMITTED	`ALTER DATABASE ... SET READ_COMMITTED_SNAPSHOT ON`
M8	Snapshot Isolation (SI)	Transaction-wide consistent snapshot	`ALLOW_SNAPSHOT_ISOLATION ON` followed by `SET TRANSACTION ISOLATION LEVEL SNAPSHOT`
M9	Version store	Storage and cleanup of row versions	`tempdb` (or PVS with ADR), automatic cleanup
M10	`READCOMMITTEDLOCK` hint, DMVs	Opt out of RCSI per table; monitor versioning	table hint; `sys.dm_tran_version_store_space_usage`
M11	Optimized locking, In-Memory OLTP	Newer lock-light / lock-free engines	SQL Server 2025 + ADR; memory-optimized tables

M1 — Deadlock Monitor (detection and resolution).

Content and purpose: SQL Server lets deadlocks happen and then breaks them, exactly the "detection and resolution" strategy of the slides. *How the engine runs it:* a background thread, the **Lock Monitor**, searches the waits-for graph of all waiting tasks every 5 seconds by default. When it finds a deadlock, the search interval drops (down to 100 ms) and goes back to 5 s once deadlocks stop. Besides lock waits, the graph also includes waits on worker threads, memory and parallel-query resources. *How to use it:* nothing to enable; the monitor is always running.

M2 — Victim selection and `SET DEADLOCK_PRIORITY`.

Content: the slides only say "one transaction is selected as victim". SQL Server applies two rules in order: (1) the session with the **lowest deadlock priority** is the victim; (2) at equal priority, the transaction that is **cheapest to roll back** (least transaction log used) is the victim; a remaining tie is broken arbitrarily. *Purpose:* protect important work (e.g. a radiologist's sign-off) and keep the rollback cost small. *How to use it:* `SET DEADLOCK_PRIORITY LOW` (−5), `NORMAL` (0, default), `HIGH` (5) or any integer from −10 to 10, per session. This also answers the slides' *starvation* problem: a session that is always sacrificed can be given a higher priority.

M3 — Error 1205 and retry.

The victim's transaction is rolled back by the engine and its client receives **Msg 1205** ("... has been chosen as the deadlock victim. Rerun the transaction."). The other transaction continues. Because the victim did nothing wrong, the standard pattern is `BEGIN TRY ... COMMIT ... END TRY BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK; ... END CATCH` plus an automatic retry when `ERROR_NUMBER() = 1205`.

M4 — `SET LOCK_TIMEOUT`.

Content and purpose: by default a session waits for a lock forever (`-1`). `SET LOCK_TIMEOUT n` makes any lock request give up after n ms with **error 1222**. It does not detect deadlocks, but it limits how long a user waits

and breaks long blocking chains (a timeout-based form of prevention, as mentioned in the slides). *Important behaviour:* error 1222 cancels only the *statement*; it does *not* roll back the transaction, and rows already sent stay sent (observed in Section 3.5.4). The application must decide to roll back or retry.

M5 — Deadlock diagnostics.

Every deadlock is written as an `xml_deadlock_report` event by the built-in Extended Events session `system_health` (always running). The XML lists the victim, every process with its `priority`, `logused`, wait resource and SQL text, and every resource with its owner and waiter. Saved as `.xdl`, it opens in SSMS as the graphical **Deadlock Graph**. Other options are trace flag 1222 (writes the same information to the error log), a custom Extended Events session, or the Deadlock graph event of SQL Profiler. Script: `01_deadlock_graph.sql`.

M6 — Avoiding deadlocks by design.

SQL Server does not implement wait-die or wound-wait, so avoidance is left to the developer: (a) access tables and rows in the **same order** in every transaction (e.g. always `Patients` then `MRI_Scans`); (b) take the **update lock U** early with the `UPDLOCK` hint when a row is read and then updated, which prevents the S→X *conversion deadlock*, because U is not compatible with U; (c) keep transactions short with no user interaction inside; (d) create indexes so that statements lock only the rows they need (the lock footprint follows the execution plan, Section 3.5.4); (e) use RCSI so that readers no longer take part in deadlocks.

M7 — Read Committed Snapshot Isolation (RCSI).

Content: a database option that changes how READ COMMITTED is implemented, with no change to the application code. *How the engine runs it:* every UPDATE/DELETE copies the previous committed version of the row into the **version store** and links it from the row through a 14-byte versioning tag (transaction sequence number + pointer). A READ COMMITTED statement then takes **no S locks**: for each row it walks the version chain back to the version committed *when the statement started*. Writers still take X locks between themselves (W–W conflicts are still serialised by locking). *Purpose:* reads never block writes and writes never block reads, and there is still no dirty read. *How to use it:* `ALTER DATABASE SpineMRI_DBMS SET READ_COMMITTED_SNAPSHOT ON WITH ROLLBACK IMMEDIATE` (needs exclusive access). This is very close to the slides' *multiversion 2PL*: readers read the committed version while a writer holds its lock, but without certify locks at commit time.

M8 — Snapshot Isolation (SI).

Content: a separate isolation level. The snapshot is taken at the **first data access of the transaction**, so all reads of the transaction see the same committed state (transaction-level, whereas RCSI is statement-level). *Concurrency rule:* if a SNAPSHOT transaction tries to modify a row that another transaction changed and committed after the snapshot started, the engine aborts it with **error 3960** ("first committer wins"). This plays the role of rule 1 of the slides' multiversion timestamp ordering: a write based on an outdated version is rejected, so no lost update is possible. *How to use it:* `ALTER DATABASE ... SET ALLOW_SNAPSHOT_ISOLATION ON`, then `SET TRANSACTION ISOLATION LEVEL SNAPSHOT` per session.

M9 — Version store and its cleanup.

Row versions live in `tempdb`; when Accelerated Database Recovery (ADR) is enabled they live in a *Persistent Version Store* inside the user database instead. A background cleanup task removes versions that no active transaction can still need. This is the "garbage collection" of the slides, and also its side effect: extra storage and I/O, plus a long-running snapshot transaction can keep many versions alive.

M10 — Fine control and monitoring.

With RCSI on, the table hint `WITH (READCOMMITTEDLOCK)` makes one statement go back to locking reads (useful when a statement must see the very latest committed value). Version activity is visible in `sys.dm_tran_version_store_space_usage` and `sys.dm_tran_active_snapshot_database_transactions`.

M11 — Other engines (not used in our database).

SQL Server 2025 offers *optimized locking* (transaction-ID locking and lock-after-qualification, built on ADR), which holds far fewer row locks for writers and therefore reduces blocking and deadlocks. Memory-optimized (In-Memory OLTP) tables use a lock-free, fully optimistic multi-version engine. Both are **off** in `SpineMRI_DBMS` (`is_optimized_locking_on = 0`, ADR off), so all results below show the classic behaviour.

Table 3.3: Chapter 5 slides versus SQL Server

Topic	Slides	SQL Server
Deadlock strategy	Prevention, detection, avoidance (wait-die, wound-wait)	Detection and resolution only (Lock Monitor); avoidance left to design (M6)
Victim choice	"a transaction of the cycle"	Lowest `DEADLOCK_PRIORITY`, then least log used
Starvation	Possible; wait-die/wound-wait avoid it	Controlled by `DEADLOCK_PRIORITY` and application retry
Waiting limit	—	`LOCK_TIMEOUT`, error 1222
Multiversion scheme	MV timestamp ordering; MV2PL with certify locks	RCSI (statement snapshot) and SI (transaction snapshot); writers keep X locks, no certify lock
Write conflict on versions	Abort T if $read_TS(X_i) > TS(T)$	SI: abort with error 3960 (first committer wins)
Old versions	Garbage collection	Version store in `tempdb` / PVS, background cleanup

3.3 Pessimistic Concurrency Analysis on Canonical Queries Q1–Q3

To evaluate engine-level locking behavior on real clinical data, we analyze the 3 Canonical Queries across the `SpineMRI_DBMS` database (575 patients, 48,345 scans, and 48,345 feature vectors) using Dynamic Management Views (`sys.dm_tran_locks` and `sys.dm_os_waiting_tasks`).

3.3.1 Canonical Query 1: Single-Patient Point Lookup and Scan Retrieval

Canonical Query Definition

Retrieves clinical notes and associated MRI scan file paths for target patient `'0001'`:

```
SELECT p.PatientID, p.PatientName, p.ClinicalNotes,
       s.ScanID, s.SeriesDescription, s.ImagePath
FROM Patients p
INNER JOIN MRI_Scans s ON p.PatientID = s.PatientID
WHERE p.PatientID = '0001';
```

Engine Locking and Plan Mechanics

- **Index Path and Cost Evaluation:** Because `PatientID` is the clustered primary key on `Patients` (`PK_Patients`), the Query Optimizer selects a point **Clustered Index Seek** (estimated cardinality = 1). On `MRI_Scans`, the foreign key index on `PatientID` permits a secondary index seek, joined via Nested Loops.
- **Lock Granularity Selected:** The Lock Manager acquires an `OBJECT: IS` lock on `Patients`, followed by a fine-grained `KEY: S` lock on the single clustered leaf entry for `PatientID = '0001'`. On `MRI_Scans`, matching scan rows are protected with `KEY: S` locks under an `OBJECT: IS` container.
- **Isolation Level Mechanics:**
  - Under default `READ COMMITTED`: The `KEY: S` lock is acquired to fetch data pages into the buffer pool and is **released immediately** upon operator completion, maximizing throughput for concurrent transactions.
  - Under `REPEATABLE READ` / `SERIALIZABLE`: The `KEY: S` lock is retained for the entire duration of the transaction. Any concurrent session attempting to modify patient `'0001'` (`UPDATE Patients ... WHERE PatientID = '0001'`) requests an incompatible `KEY: X` lock and is placed into a `SUSPENDED` state in the FIFO queue until commit.

Results Messages

	Resource	LockMode	EntityName	IndexName	LockCount
1	DATABASE	S	SpineMRI_DBMS	N/A	2
2	OBJECT	IS	MRI_Scans	N/A	1
3	OBJECT	IS	Patients	N/A	1
4	PAGE	IS	MRI_Scans	PK_MRI_Scans	1874
5	PAGE	IS	Patients	PK_Patients	1
6	KEY	S	MRI_Scans	PK_MRI_Scans	65
7	KEY	S	Patients	PK_Patients	1

Figure 3.3: Dynamic Management View (`sys.dm_tran_locks`) Lock Verification for Query 1 under `REPEATABLE READ`

3.3.2 Canonical Query 2: Filtered Scan Retrieval by Pulse Sequence (T2 Sagittal)

Canonical Query Definition

Retrieves diagnostic narrative and scan paths restricted to T2 sagittal sequences (`%T2%SAG%`) for a designated patient:

```
SELECT p.PatientID, p.PatientName, p.ClinicalNotes,
       s.ScanID, s.SeriesDescription, s.ImagePath
FROM Patients p
INNER JOIN MRI_Scans s ON p.PatientID = s.PatientID
WHERE p.PatientID = '0001'
  AND s.SeriesDescription LIKE '%T2%SAG%';
```

Engine Locking and Phantom Read Dynamics

- **Predicate Processing and Filter Locks:** The engine performs an exact Clustered Index Seek on `Patients` for `'0001'`, followed by a join to `MRI_Scans`. Because the string predicate contains a leading wildcard (`%T2%SAG%`), the engine evaluates the non-key attribute filter on candidate rows matching `PatientID = '0001'`.
- **Phantom Read Vulnerability under REPEATABLE READ:** When executed inside a `REPEATABLE READ` transaction, SQL Server places `KEY: S` locks on all existing scan records for patient `'0001'`. However, if concurrent Session 2 inserts a *new* T2 sagittal slice for patient `'0001'` and commits, Session 1's subsequent re-execution observes the newly inserted slice, suffering a **Phantom Read**.
- **Phantom Prevention under SERIALIZABLE:** Under `SERIALIZABLE`, the Lock Manager applies `RangeS-S` key-range locks covering the index key interval for `PatientID = '0001'` on the B-Tree leaf. If Session 2 attempts to insert a new scan row for patient `'0001'`, the engine requests a `RangeI-N` lock that conflicts with `RangeS-S`. Session 2 is blocked immediately until Session 1 finishes, eliminating phantom reads.

Results Messages

	BlockedSessionID	WaitDuration_MS	WaitType	BlockingSessionID	ResourceType	LockModeHeldBySession1
1	100	9767	LCK_M_IX	76	DATABASE	S
2	100	9767	LCK_M_IX	76	PAGE	IS
3	100	9767	LCK_M_IX	76	OBJECT	S
4	100	9767	LCK_M_IX	76	OBJECT	IS
5	100	9767	LCK_M_IX	76	KEY	S

Figure 3.4: Key-Range Locking (`RangeS-S`) and Phantom Insertion Blocking under `SERIALIZABLE` in Query 2

3.3.3 Canonical Query 3: AI Feature Similarity Retrieval (Content-Based Search)

Canonical Query Definition

Calculates mathematical Euclidean distance across normalized AI feature embeddings (`V_Dim1..V_Dim4`) over all 48,345 slices to retrieve top-3 visually similar scans:

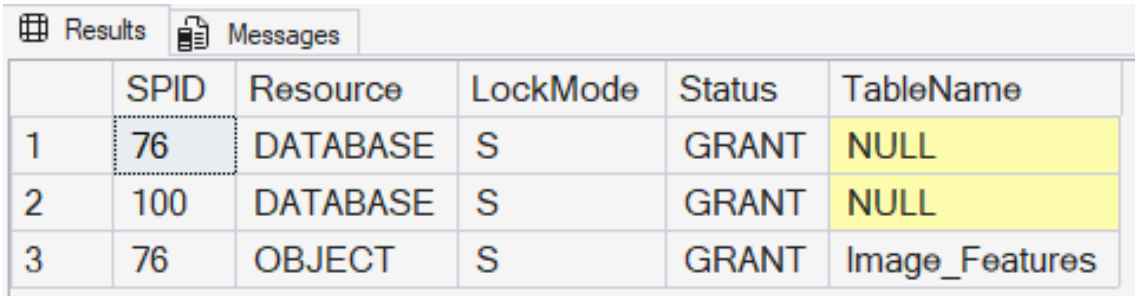
```
DECLARE @TargetScanID INT = 1;
DECLARE @T1 FLOAT, @T2 FLOAT, @T3 FLOAT, @T4 FLOAT;

SELECT @T1 = V_Dim1, @T2 = V_Dim2, @T3 = V_Dim3, @T4 = V_Dim4
FROM Image_Features WHERE ScanID = @TargetScanID;

SELECT TOP 3 s.ScanID, s.PatientID, s.SeriesDescription, s.ImagePath,
    SQRT(SQUARE(f.V_Dim1 - @T1) + SQUARE(f.V_Dim2 - @T2) +
         SQUARE(f.V_Dim3 - @T3) + SQUARE(f.V_Dim4 - @T4)) AS EuclideanDistance
FROM Image_Features f
INNER JOIN MRI_Scans s ON f.ScanID = s.ScanID
WHERE f.ScanID <> @TargetScanID
ORDER BY EuclideanDistance ASC;
```

Engine Locking and Lock Escalation Dynamics

- **Full Table Scan Requirements:** Because calculating non-linear Euclidean distance ($\sqrt{\sum(u_k - v_k)^2}$) requires computing arithmetic differences across every row, standard B-Tree indexes cannot filter the calculation via seeks. The Query Optimizer executes a **Clustered Index Scan** over all 48,345 rows in `Image_Features`, followed by a `Top N Sort`.
- **Lock Granularity Traversal:** During the scan phase under default `READ COMMITTED`, SQL Server places an `OBJECT: IS` lock on `Image_Features` and traverses 8KB pages, acquiring and releasing `PAGE: S` locks in rapid succession. Because locks are dropped as pages are processed, memory consumption remains negligible.
- **Lock Escalation Exposure on the Ingested Clinical Database:** On the full production database of 48,345 scans, if this query is executed within an open transaction under `REPEATABLE READ` or `SERIALIZABLE`, the engine must retain shared locks on every accessed page. When the number of accumulated locks crosses the 5,000 threshold, the Lock Manager triggers **Lock Escalation**, converting thousands of fine-grained page/key locks into a single `OBJECT: S` table lock. This reclaims memory in the buffer pool, but blocks all concurrent transactional modifications (`INSERT`, `UPDATE`) on the entire `Image_Features` table.


Results Messages

	SPID	Resource	LockMode	Status	TableName
1	76	DATABASE	S	GRANT	NULL
2	100	DATABASE	S	GRANT	NULL
3	76	OBJECT	S	GRANT	Image_Features

Figure 3.5: Execution Plan Operators and Lock Escalation Footprint for Query 3 Vector Distance Search

3.4 Synthesis: ANSI SQL Isolation Levels vs. MSSQL Engine Mechanics

Table 3.4 synthesizes theoretical isolation levels, potential data anomalies, and Microsoft SQL Server's concrete engine-level locking behavior across the clinical imaging workload.

Table 3.4: ANSI SQL Isolation Levels and Concrete MSSQL Engine Locking Behaviors

Isolation Level	Dirty Read	Non-repeatable Read	Phantom Read	Concrete MSSQL Engine Locking Behavior
READ UNCOMMITTED	Yes	Yes	Yes	Executes reads without acquiring S-locks; ignores active X-locks held by concurrent writers.
READ COMMITTED	No	Yes	Yes	Acquires fine-grained S-locks, but releases them immediately after statement execution.
REPEATABLE READ	No	No	Yes	Retains fine-grained S-locks on accessed keys/pages until transaction termination.
SERIALIZABLE	No	No	No	Applies **Key-Range Locks** (`RangeS-S`) on B-Tree intervals to prevent phantom insertions.

3.5 Empirical Deadlock Resolution and MVCC Evaluation

3.5.1 Experimental Environment and Diagnostic Harness

- **Engine:** Microsoft SQL Server 2025 (RTM) 17.0.1000.7 Developer Edition, compatibility level 170; database restored from `data/SpineMRI_DBMS.bak` (575 patients, 48,345 scans). ADR, optimized locking, RCSI and SNAPSHOT are OFF at the start.
- **Two sessions:** each scenario has a Session A file and a Session B file in `scripts/sql/role5/`, opened in two SSMS windows. The session that holds locks contains a `WAITFOR DELAY` so the presenter can

switch window. A third window may run `03_lock_monitor.sql`, which shows the waits-for edges (`sys.dm_exec_requests.blocking_session_id`) and the lock table (`sys.dm_tran_locks`).

- **Measurements:** *elapsed time* of the statement, and *lock wait time* taken from `sys.dm_exec_session_wait_stats` (sum of `LCK_M_*` waits of the session, before/after the statement). The number of locks a query requests per table comes from `sys.dm_db_index_operational_stats` (`row_lock_count`, `page_lock_count`). For the reported numbers the same files were launched as parallel `sqlcmd` sessions (B started 2 s after A); raw logs are in `scripts/sql/role5/evidence/run_logs/`. `99_reset.sql` restores the data after each run.

3.5.2 Experiment 1: Deadlock Induction, Victim Selection, and Deadlock Graph Analysis

Scenario. Two staff members update the data that Query 1 returns, in opposite order. Session A (radiologist) updates patient `0001`, then the slice `ScanID = 1` of that patient; Session B (technician) updates slice `1`, then patient `0001`. Both use `TRY...CATCH` with `ROLLBACK`.

```
-- Session A (01_deadlock_A.sql)                -- Session B (01_deadlock_B.sql)
SET DEADLOCK_PRIORITY NORMAL;  -- run 2: LOW    SET DEADLOCK_PRIORITY NORMAL;
BEGIN TRANSACTION;                              BEGIN TRANSACTION;
UPDATE dbo.Patients SET DiagnosisStatus =       UPDATE dbo.MRI_Scans
  'Under Review' WHERE PatientID = '0001';        SET AcquisitionDate = SYSDATETIME()
WAITFOR DELAY '00:00:06';                         WHERE ScanID = 1;
UPDATE dbo.MRI_Scans                            WAITFOR DELAY '00:00:02';
  SET AcquisitionDate = SYSDATETIME()           UPDATE dbo.Patients SET DiagnosisStatus =
  WHERE ScanID = 1;          -- waits for B       'Image Checked' WHERE PatientID = '0001';
COMMIT TRANSACTION;                             COMMIT TRANSACTION;  -- waits for A
```

The lock monitor captured the cycle of the waits-for graph before the Lock Monitor broke it (run 1, session 62 = A, 63 = B):

```
WaitingSession BlockedBy wait_type WaitMs WaitingStatement
62             63        LCK_M_X   270    UPDATE [dbo].[MRI_Scans] ... WHERE [ScanID]=@1
63             62        LCK_M_X   2236   UPDATE [dbo].[Patients]  ... WHERE [PatientID]=@2

SessionID Resource ObjectName IndexName     KeyHash         Mode Status
62        KEY      Patients   PK_Patients   (5f6f76815dbe)  X    GRANT
62        KEY      MRI_Scans  PK_MRI_Scans  (8194443284a0)  X    WAIT
63        KEY      MRI_Scans  PK_MRI_Scans  (8194443284a0)  X    GRANT
63        KEY      Patients   PK_Patients   (5f6f76815dbe)  X    WAIT
```

The edges 62 → 63 and 63 → 62 form exactly the cycle of the slides' wait-for graph. Table 3.5 gives the outcome of two runs, using the values read from the captured `xml_deadlock_report` events.

Table 3.5: Experiment 1: victim selection (values from the deadlock graph XML)

Run	Session A		Session B		Victim	Cycle → resolved
	priority	log used (bytes)	priority	log used (bytes)		
1: NORMAL / NORMAL	0	308	0	272	B (error 1205)	1.83 s
2: LOW / NORMAL	−5	308	0	272	A (error 1205)	1.21 s

Analysis. In run 1 both priorities are equal, so SQL Server applies the second rule and sacrifices B, which has written less log (272 vs. 308 bytes) and is cheaper to roll back. In run 2 A has a lower priority, and A becomes the victim *even though it is the more expensive one to roll back*: priority is checked before cost. In both runs the victim received Msg 1205, its update was undone, and the survivor's blocked statement was granted and committed. The cycle was broken 1.2–1.8 s after it formed, which fits the 5-second search interval of the Lock Monitor. Figure 3.6 redraws the graph of run 1 from `evidence/deadlock_run1_priority_normal.xdl`; Figures 3.7 and 3.8 are the graphs of run 1 and run 2 rendered by SSMS.

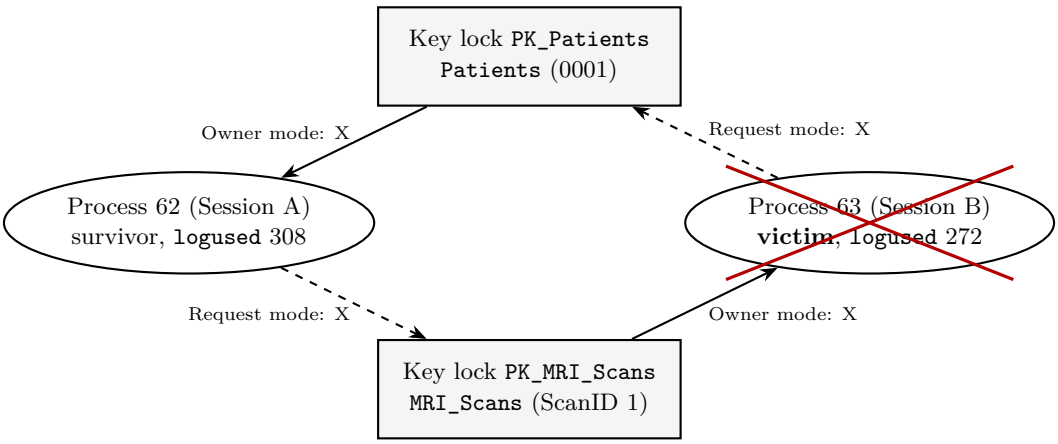


Figure 3.6: Deadlock graph of run 1, redrawn from the `xml_deadlock_report` of `system_health`. Solid arrows: lock owned; dashed arrows: lock requested. The two requests close the cycle; the crossed process is the victim.

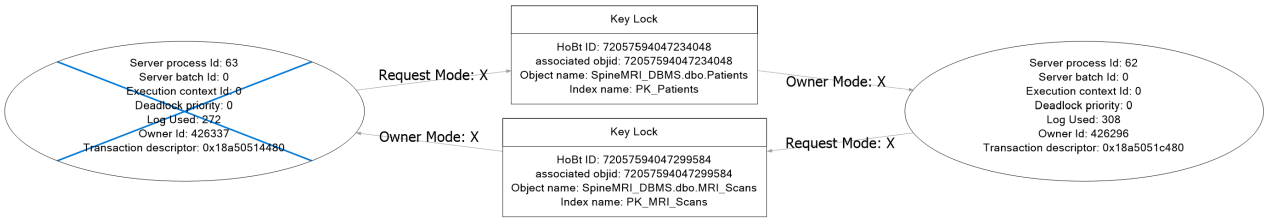


Figure 3.7: Deadlock Graph rendered by SSMS from `deadlock_run1_priority_normal.xdl`. Ovals are processes (sessions), rectangles are the locked resources (key locks on `PK_Patients` and `PK_MRI_Scans`). An arrow from a resource to a process means "Owner Mode: X" (the process holds it); an arrow from a process to a resource means "Request Mode: X" (the process waits for it). Process 63 (Session B, Log Used 272) is crossed out as the victim; process 62 (Session A, Log Used 308) survives; both have Deadlock priority 0.

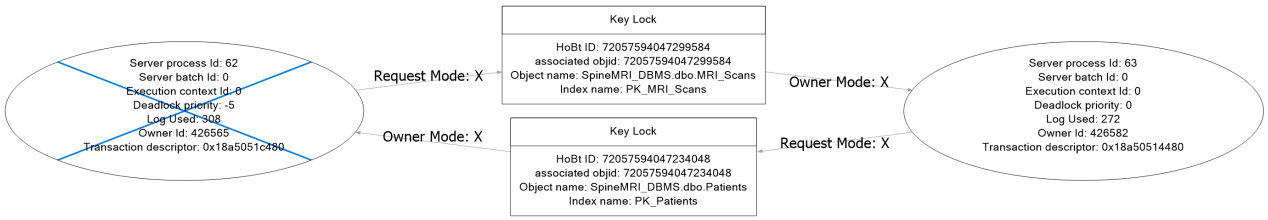


Figure 3.8: Deadlock Graph of run 2 (`deadlock_run2_A_priority_low.xdl`): the same cycle, but Session A now runs with `SET DEADLOCK_PRIORITY LOW`. Process 62 (Session A, Deadlock priority −5, Log Used 308) is crossed out as the victim, although it is the *more* expensive transaction to roll back; process 63 (Session B, priority 0, Log Used 272) survives. Priority is evaluated before rollback cost.

3.5.3 Experiment 2: Lock Wait Time Benchmarking — Classic Locking versus RCSI

Scenario. Session A updates the diagnosis of patient `0001` and keeps the transaction open for 10 s (X lock held). Session B reads the same row at READ COMMITTED. The same pair of scripts is run three times with RCSI OFF and three times with RCSI ON (`02_rcsi_0_enable_row_versioning.sql`).

```
-- Session A (02_rcsi_A_writer.sql)
BEGIN TRANSACTION;
    UPDATE dbo.Patients SET DiagnosisStatus = 'Revised' WHERE PatientID = '0001';
    WAITFOR DELAY '00:00:10';
COMMIT TRANSACTION;

-- Session B (02_rcsi_B_reader.sql)
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT @v = DiagnosisStatus FROM dbo.Patients WHERE PatientID = '0001';
-- + elapsed time and LCK_M_* wait time from sys.dm_exec_session_wait_stats
```

Table 3.6: Experiment 2: lock wait time of the reader, same scenario, 3 runs each

Mode	Run	Value read	Elapsed (ms)	Lock wait (ms)	Wait type
Locking READ COMMITTED (RCSI OFF)	1	`Revised`	7,931	7,930	`LCK_M_S`
	2	`Revised`	7,931	7,930	`LCK_M_S`
	3	`Revised`	7,953	7,953	`LCK_M_S`
READ COMMITTED SNAPSHOT (RCSI ON)	1	`Confirmed`	0	0	—
	2	`Confirmed`	0	0	—
	3	`Confirmed`	0	0	—
Average locking / RCSI			**7,938 / 0**	**7,938 / 0**	

Analysis. With classic locking, B's S request on key `(5f6f76815dbe)` is incompatible with A's X lock; the monitor shows `X GRANT` for A and `S WAIT` for B, and B waits until A commits (about 8 s, because B started 2 s after A), then reads the new value `Revised`. With RCSI the lock table during the same moment contains *only* A's `IX/IX/X` locks — B requests no S lock at all — and `sys.dm_tran_version_store_space_usage` reports **8 pages (64 KB)** of row versions. B immediately reads `Confirmed`, the last *committed* version taken from the version store. Neither mode ever returns an uncommitted value. The trade-off: RCSI removes the waiting but may return a version that is a few seconds older.

Snapshot Isolation (`02_snapshot_B_reader.sql`). A SNAPSHOT transaction read patient `0001` at the start (0 ms, `Confirmed`), waited while A committed `Revised`, and read again: the second read still returned `Confirmed`, while the committed value was already `Revised`. This shows the transaction-level snapshot of SI, versus the statement-level snapshot of RCSI.

Update conflict (`02_snapshot_C_update_conflict.sql`). A SNAPSHOT transaction read the row, A updated and committed it, and the snapshot transaction then tried to update it. SQL Server refused with **error 3960** ("Snapshot isolation transaction aborted due to update conflict ..."). Under versioning, a write–write conflict is therefore resolved by aborting rather than by waiting, and no lost update can happen.

3.5.4 Experiment 3: Canonical Queries Q1–Q3 under Active Concurrent Writers

This experiment applies the mechanisms above to the three canonical queries. Session A (`04_queries_A_writer.sql`) re-labels one slice of **patient `0002`** (`ScanID = 80`) and keeps its X locks for 10 s before rolling back. Patient `0002` is *not* in the result of Q1 or Q2 (which ask for patient `0001`) and is one of the 48,345 rows that Q3 compares. Session B runs Q1, Q2 or Q3 unchanged (`04_queries_B_Q1/Q2/Q3.sql`). Every combination of two index states (*PK only*, as in the backup; *PK + covering*, with Role 2's index `IX_MRIScans_PatientID_Covering (PatientID, SeriesDescription) INCLUDE (ImagePath)`) and two modes (locking, RCSI) was run, each query once alone (lock footprint) and once against the writer (blocking).

Table 3.7: Access path chosen by the optimizer (estimated plans, `SHOWPLAN_TEXT`)

Query	PK only	PK + covering index
Q1	Nested Loops: Clustered Index **Seek** `PK_Patients` + Clustered Index **Scan** `PK_MRI_Scans` (filter `PatientID='0001'`)	Nested Loops: Clustered Index Seek `PK_Patients` + Index **Seek** `IX_MRIScans_PatientID_Covering`
Q2	Same as Q1, plus the `LIKE '%T2%SAG%'` filter in the scan	Same as Q1; `LIKE` becomes a residual predicate of the seek
Q3	Parallel Hash Match: Clustered Index **Scan** `PK_Image_Features` + Clustered Index **Scan** `PK_MRI_Scans`, then Top-N Sort	Same, but `MRI_Scans` is read by an Index **Scan** of the narrower covering index

Table 3.8: Concurrency behaviour of the canonical queries while a writer holds X locks on one row of patient 0002. Locks requested by the query alone at locking READ COMMITTED: r = row locks, p = page locks, (P) = `Patients`, (F) = `Image_Features`. Under RCSI every query requested 0 locks. Times in ms.

Index	Q	Locks requested		Locking RC		RCSI	
		MRI_Scans	Other table	Elapsed	Lock wait	Elapsed	Lock wait
PK only	Q1	1,874 p	1 r + 1 p (P)	7,976	7,969	5	0
	Q2	1,874 p	1 r + 1 p (P)	7,982	7,976	7	0
	Q3	1,903 p	1 r + 3,780 p (F)	7,983	7,972	16	0
PK + covering	Q1	65 r + 3 p	1 r + 1 p (P)	1	0	1	0
	Q2	65 r + 3 p	1 r + 1 p (P)	1	0	1	0
	Q3	1,588 p	1 r + 3,780 p (F)	7,993	7,982	17	0

How SQL Server decides, query by query.

- **The optimizer chooses the plan by cost, and the plan decides the lock footprint.** The Query Optimizer does not consider concurrency; it picks the cheapest access path (Table 3.7). At run time the storage engine then chooses the **lock granularity from the access method**: for a *scan* it requests **page locks** (one lock per 8 KB page instead of one per row, e.g. 1,874 page locks and 0 row locks on `MRI_Scans`), for a *seek* it requests **row (key) locks** plus intent locks on the few pages visited (65 key locks + 3 page locks). Under locking READ COMMITTED each S lock is released right after the row/page is read, so no lock escalation occurs even for Q3.
- **Q1 and Q2 with PK only: blocked by a patient they do not return.** Without an index on `MRI_Scans.PatientID`, Q1 and Q2 must scan the whole table and test every row. When the scan reaches the page of `ScanID 80`, its page S lock conflicts with the writer's IX/X locks, so the query waits ≈8 s for a row of patient `0002`. With `LOCK_TIMEOUT 3000` (`01_lock_timeout_B_Q1.sql`) the same Q1 had already streamed all 65 rows of patient `0001` (they are on the first pages) and then failed with error 1222 after 3,008 ms: SQL Server cannot know there are no more `0001` rows until the scan is finished.
- **Q1 and Q2 with the covering index: no blocking at all, even under locking.** The seek visits only the key range `PatientID = '0001'` of the index, so it never touches the locked row of patient `0002`. A good index therefore reduces blocking and deadlock risk as much as it reduces I/O. For Q2 the `LIKE '%T2%SAG%'` test is a residual predicate: the 65 keys of the patient are still locked and read, then 15 rows are returned.
- **Q3: always blocked under locking.** Q3 must compare the reference vector with every other slice, so both tables are scanned completely (3,780 page locks on `Image_Features`, 1,588–1,903 on `MRI_Scans`). Any writer anywhere in `MRI_Scans` — here a re-label that also changes the covering index key — blocks it. Adding indexes cannot help a query that needs all rows; only versioning can.
- **RCSI: zero locks, zero waiting, for all three queries.** With RCSI ON the three queries requested **no row or page locks** on any table and never waited (0 ms lock wait in both index states), while still returning only committed data. This also removes the readers from every possible deadlock: a statement that holds no S lock cannot be part of a waits-for cycle.
- **Deadlock exposure.** At locking READ COMMITTED a reader holds at most the lock of the current row/page, so Q1–Q3 alone rarely form a cycle. Deadlocks appear between *writers* that maintain the data the queries read, as in Experiment 1 (the record and a slice of patient `0001`, i.e. Q1's result). SQL Server resolves them with M1–M3; the design rules of M6 prevent them.

3.6 Chapter Synthesis and Comprehensive Recommendations

- SQL Server resolves deadlocks by **detection**: the Lock Monitor found every cycle within 1.2–1.8 s and rolled back the victim with Msg 1205. The victim is chosen by `DEADLOCK_PRIORITY` first and log cost second, as both runs show; this is the tool to protect critical transactions and to avoid starvation.
- **RCSI** cut the reader's lock wait from 7,938 ms (average of 3 runs) to 0 ms in the same scenario, for a version store cost of 64 KB. For the canonical queries it removed all S locks (up to 5,683 page locks for Q3) and all waiting. It is the recommended setting for this read-heavy clinical database (doctors browsing reports and images), with `READCOMMITTEDLOCK` for the few statements that need the very latest value.
- **Snapshot Isolation** is suited to long reports that need one consistent picture across several statements; the application must retry on error 3960.
- For writers: access `Patients` before `MRI_Scans` in every transaction, keep transactions short, retry on 1205, use `LOCK_TIMEOUT` where users must not wait indefinitely, and index the foreign key `MRI_Scans.PatientID` so that Q1/Q2 lock 65 keys instead of 1,874 pages.